

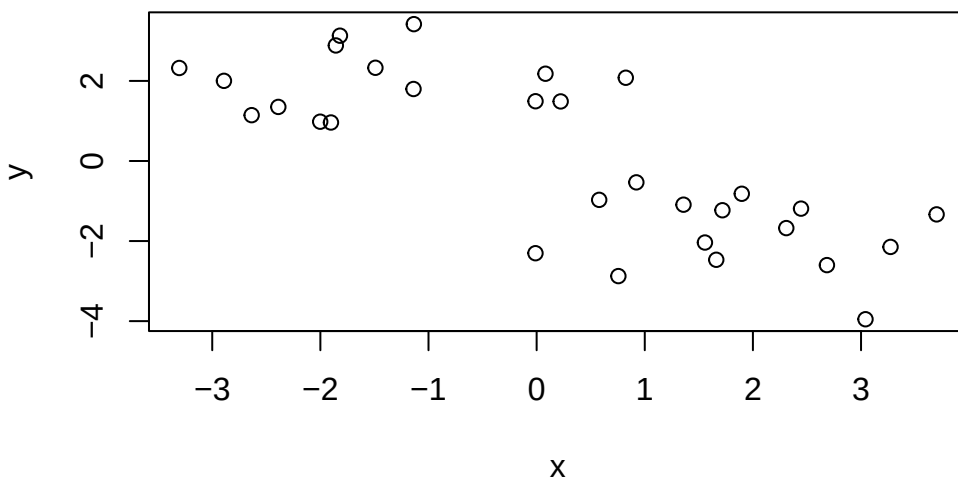
Class07

Lea Sounthong

Generate some data

```
#Generate some example data  
tmp_x <-c(rnorm(15, -2), rnorm(15, 2))  
tmp_y <- c(rnorm(15, 2), rnorm(15, -2))  
df<- cbind(x=tmp_x, y=tmp_y)
```

```
#Plotting the data  
plot(df)
```



```
#Lets do a k-means clustering analysis
km <- kmeans(df, centers = 2, nstart = 20)
km
```

K-means clustering with 2 clusters of sizes 15, 15

Cluster means:

	x	y
1	-1.429770	1.971137
2	1.859259	-1.815095

Clustering vector:

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

Within cluster sum of squares by cluster:

```
[1] 29.33706 27.63305
(between_SS / total_SS = 76.8 %)
```

Available components:

[1] "cluster"	"centers"	"totss"	"withinss"	"tot.withinss"
[6] "betweenss"	"size"	"iter"	"ifault"	

Class Q. How many points are in each cluster?

```
km$size
```

```
[1] 15 15
```

There are 15 points in cluster 1 and 15 points in cluster 2.

What ‘component’ of your result object details:

Class Q. Cluster size?

```
km$size
```

```
[1] 15 15
```

Class Q. Cluster assignment?

```
km$cluster
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

```
table(km$cluster)
```

```
1 2  
15 15
```

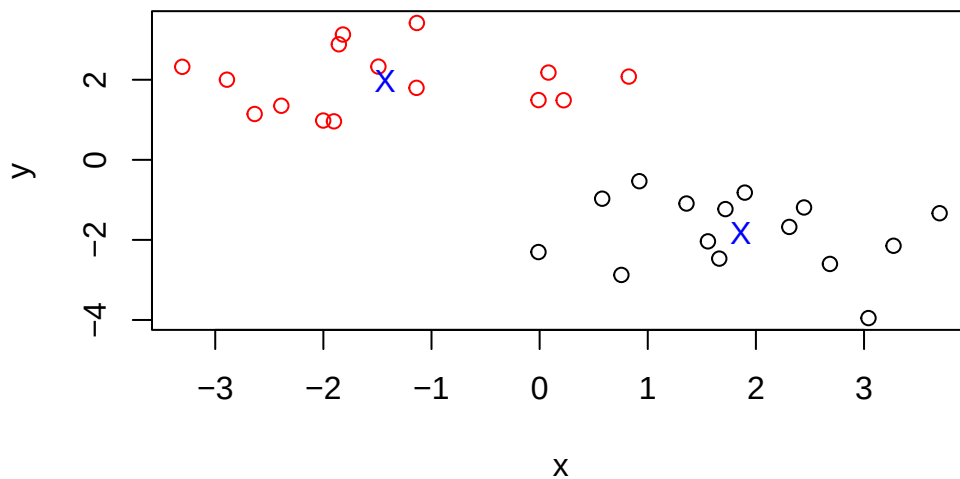
Q. Cluster Center?

```
km$centers
```

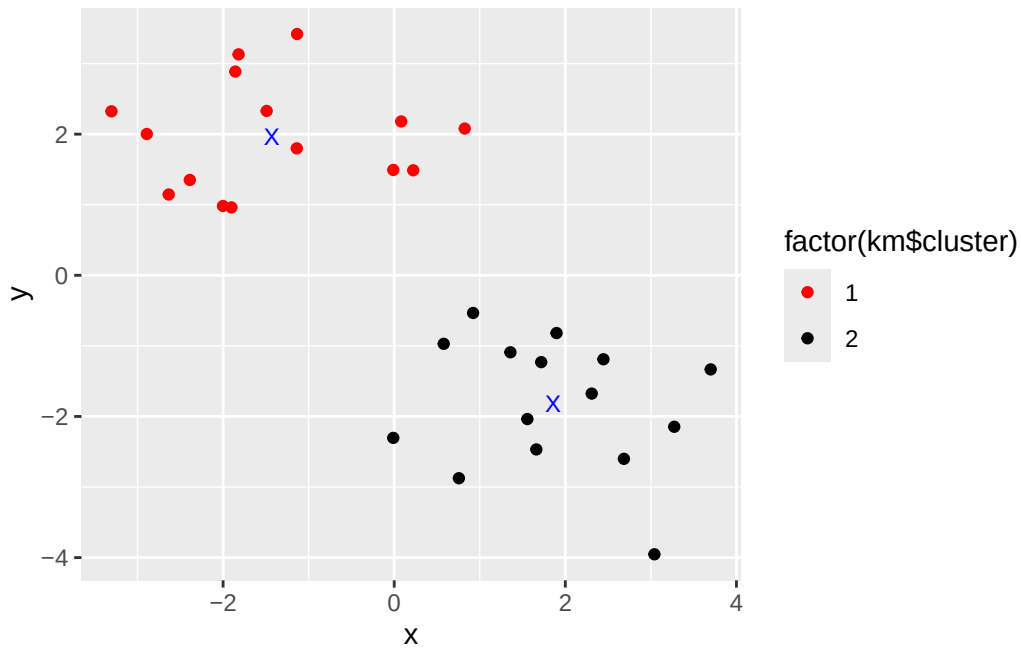
```
      x      y  
1 -1.429770  1.971137  
2  1.859259 -1.815095
```

Plot x colored by kmeans cluster assignment

```
#Plotting k-means clustering  
plot(df, col=c("RED", "BLACK")[km$cluster])  
points(km$centers, col="BLUE", pch="X")
```



```
#Generating a ggplot
library(ggplot2)
ggplot(df) +
  aes(x, y, col=factor(km$cluster)) +
  geom_point() +
  scale_color_manual(values = c("RED", "BLACK")) +
  geom_point(data = km$centers, col="BLUE", shape = "X", size = 3, aes(x, y))
```



Hierarchical Clustering

Analyze by hclust

```
dm <- dist(df)
```

```
#dm
```

```
hc <- hclust(dm)
```

```
hc
```

Call:

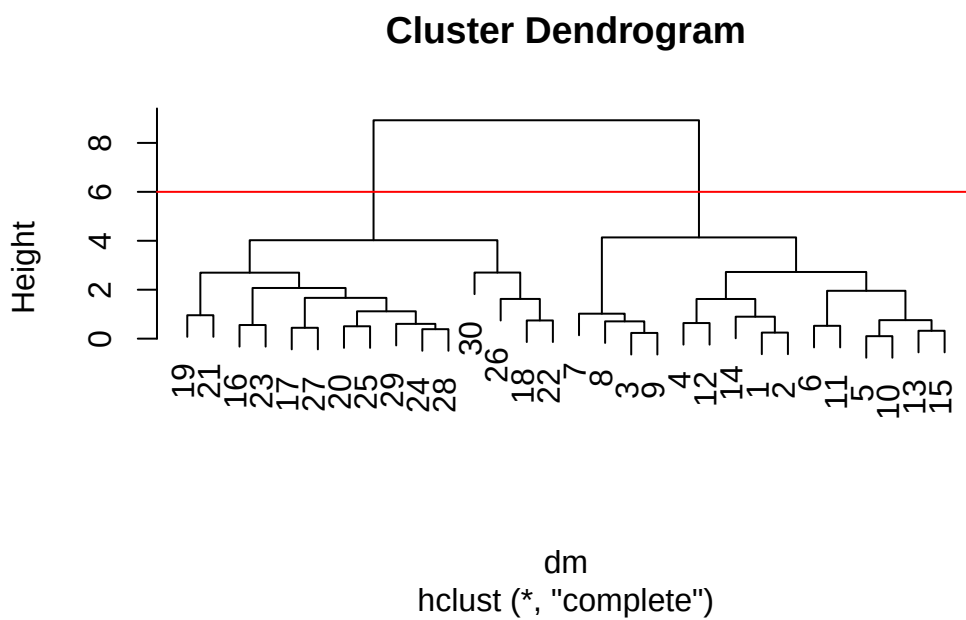
```
hclust(d = dm)
```

Cluster method : complete

Distance : euclidean

Number of objects: 30

```
plot(hc)
abline(h=6, col="RED")
```



Let's use cutree:

Cut by height:

```
cutree(hc, h=6)
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

```
table(cutree(hc, h=6))
```

```
1 2
15 15
```

Cut by k:

```
cutree(hc, k=2)
```

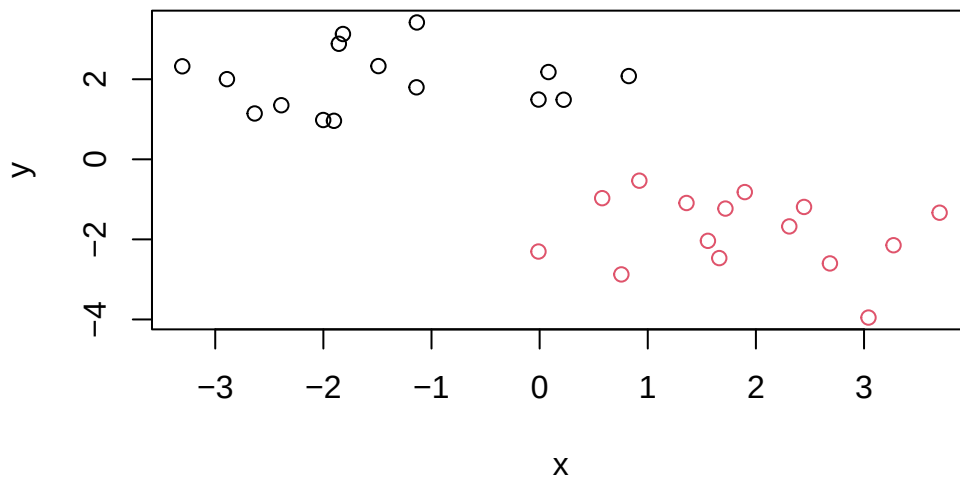
```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

4 clusters

```
cutree(hc, k=4)
```

```
[1] 1 1 2 1 1 1 2 2 2 1 1 1 1 1 1 3 3 4 3 3 3 4 3 3 3 4 3 3 3 4
```

```
plot(df, col=cutree(hc, k=2))
```



PCA Lab Sheet:

```
#Importing data from UK_foods.csv input file
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
```

Q1. How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

```
#Finding out how many rows and columns are in x
dim(x)
```

```
[1] 17  5
```

Checking your data:

```
#Preview the first 6 rows
head(x)
```

	X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66
2	Carcass_meat	245	227	242	267
3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139

```
#Seeing how the minus indexing works
rownames(x) <- x[,1]
x <- x[,-1]
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

```
#Running dimensions again
dim(x)
```

```
[1] 17  4
```

```
#Generating an alternative approach to setting correct row names
x <- read.csv(url, row.names=1)
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

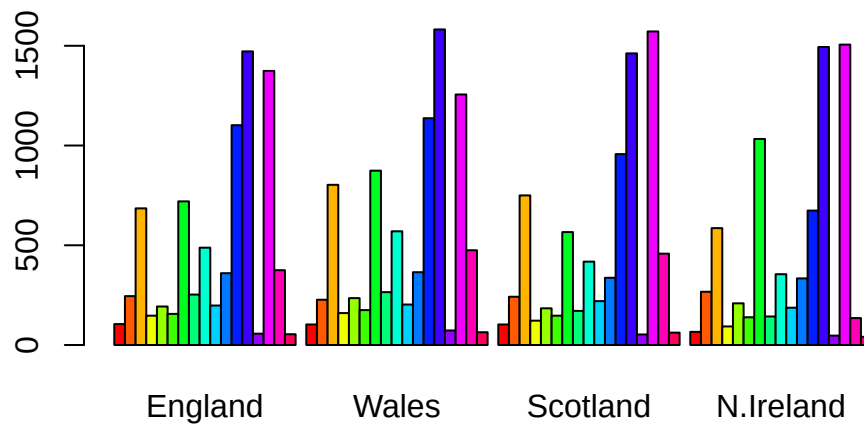
There are 17 rows and 4 columns.

Q2: Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

I would prefer to use the `read.csv(url, row.names=1)` because it is cleaner and sets the row names correctly from the start. When using the minus indexing method, if you run it multiple times, it keeps removing the first column, which can remove important data.

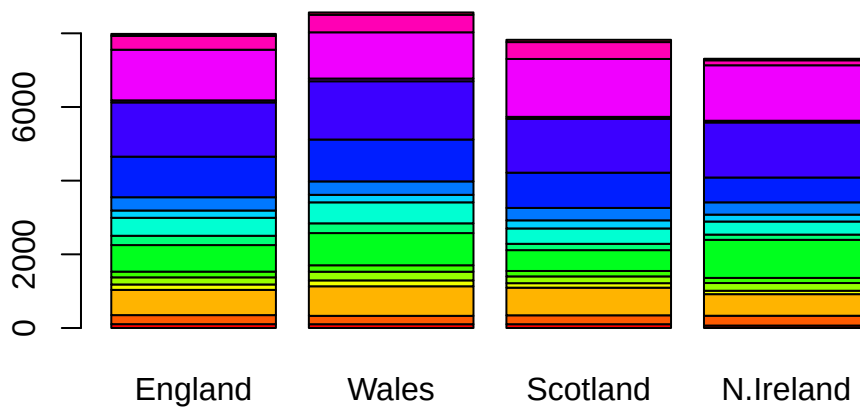
Spotting major differences and trends:

```
#Using base R
barplot(as.matrix(x), beside = T, col = rainbow(nrow(x)))
```

Q3: Changing what optional argument in the above `barplot()` function results in the following plot?

```
#Using base R but changing beside to FALSE
barplot(as.matrix(x), beside = F, col = rainbow(nrow(x)))
```



Side-note: Using ggplot and the need for “tidy” data

```
#loading in data from tidyr package
library(tidyr)
# Convert data to long format for ggplot with `pivot_longer()`
x_long <- x |>
  tibble::rownames_to_column("Food") |>
  pivot_longer(cols = -Food,
               names_to = "Country",
               values_to = "Consumption")
dim(x_long)
```

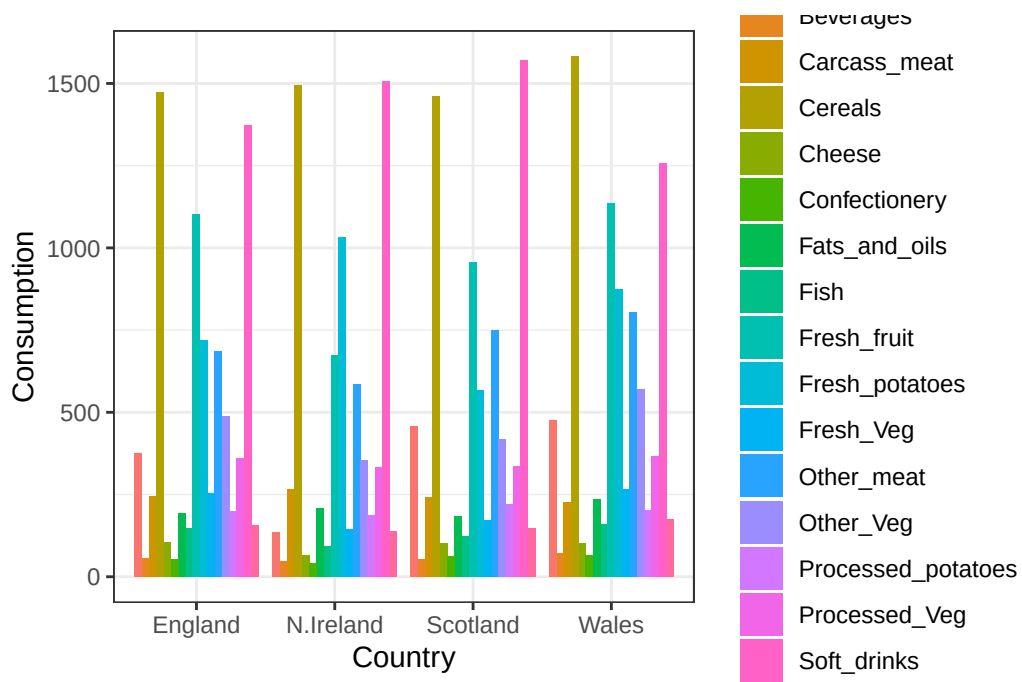
```
[1] 68  3
```

```
#Preview the first 6 rows of x_long
head(x_long)
```

```
# A tibble: 6 x 3
  Food      Country Consumption
  <chr>    <chr>         <int>
1 "Cheese" England         105
2 "Cheese" Wales           103
```

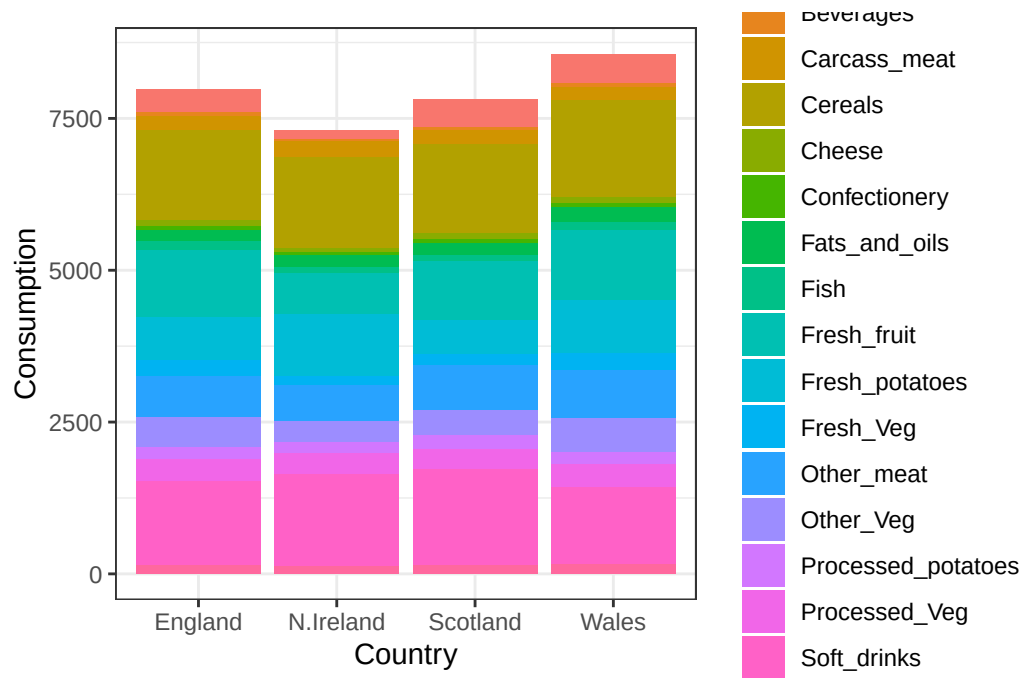
3	"Cheese"	Scotland	103
4	"Cheese"	N.Ireland	66
5	"Carcass_meat "	England	245
6	"Carcass_meat "	Wales	227

```
#Generating grouped bar plot
library(ggplot2)
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_bw()
```



Q4: Changing what optional argument in the above ggplot() code results in a stacked barplot figure?

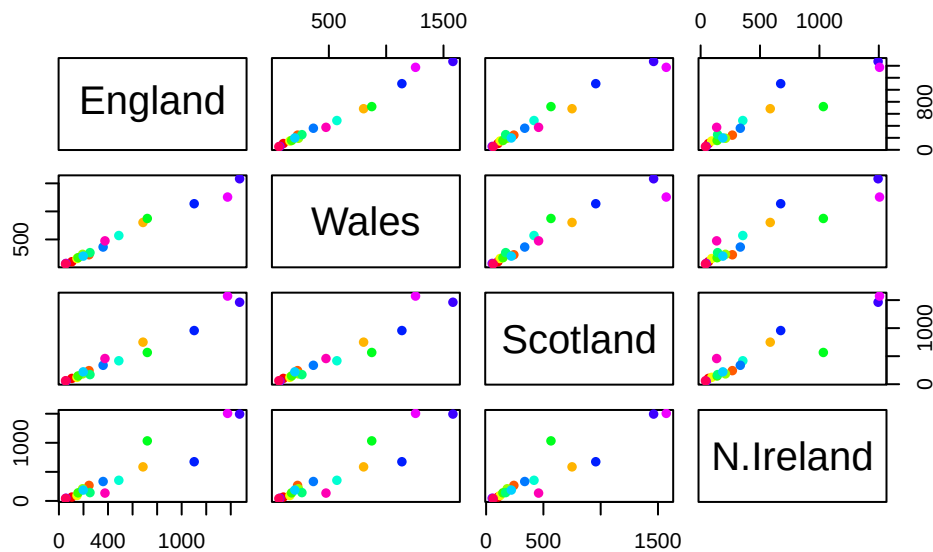
```
#Generating stacked barplot figure
library(ggplot2)
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "stack") +
  theme_bw()
```



Pairs plots and heatmaps

Q5: We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

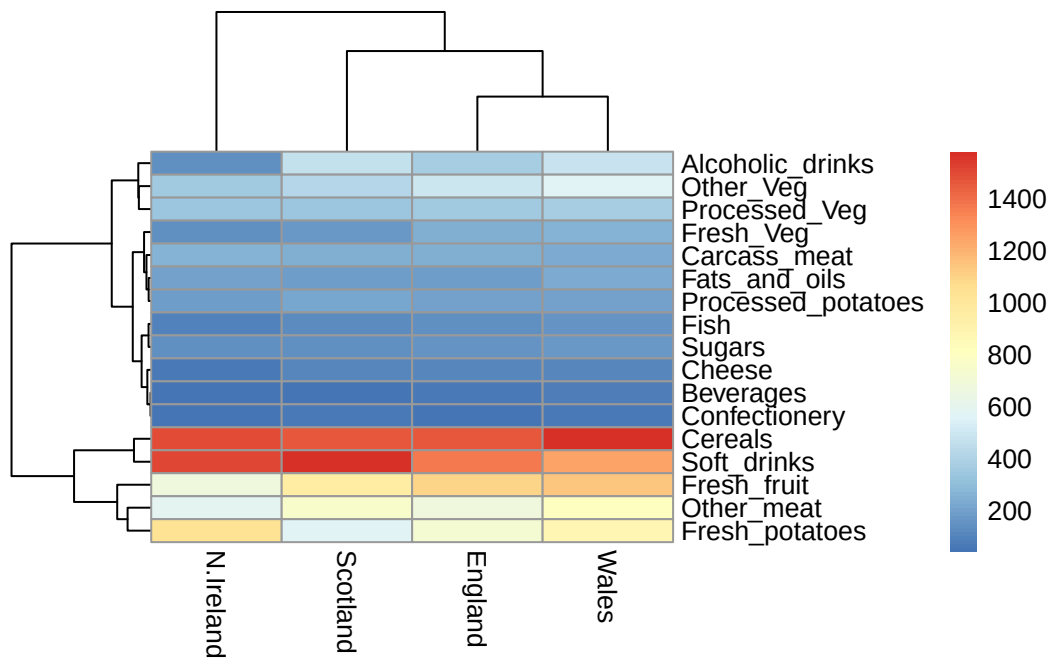
```
#Generating all pairwise plots for our countries
pairs(x, col=rainbow(nrow(x)), pch = 16)
```



The `pairs()` function generates scatter plots that compare every pair of variables, in this case countries, and compare the values. Each point represents a food category, however a legend would be needed for each color to be matched to a specific food category. The colors are assigned using `col=rainbow`, (`nrow(x)`) creates that many different colors so each row has a unique color, and the `pch=16` function sets the points to be solid filled circles. If a point lies on the diagonal, it means the two countries being compared have the same consumption value for that food category.

```
#Generating a heatmap to make it easier to spot patterns
#and differences from the color scale
library(pheatmap)

pheatmap( as.matrix(x) )
```



Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

Based on the pairs and heatmap figures, England and Wales cluster closely together, suggesting that these countries are closer in similarity when it comes to food consumption patterns. Scotland has fairly clustered with England and Wales, so it has somewhat similar food consumption patterns to England and Wales. From this data set, it seems like N.Ireland has less consumption fresh fruit and other meat compared to the other countries in the UK. However, N.Ireland seems to have a higher consumption of fresh potatoes compared to the other countries.

```
#Use the prcomp() PCA functon
pca <- prcomp(t(x))
summary (pca)
```

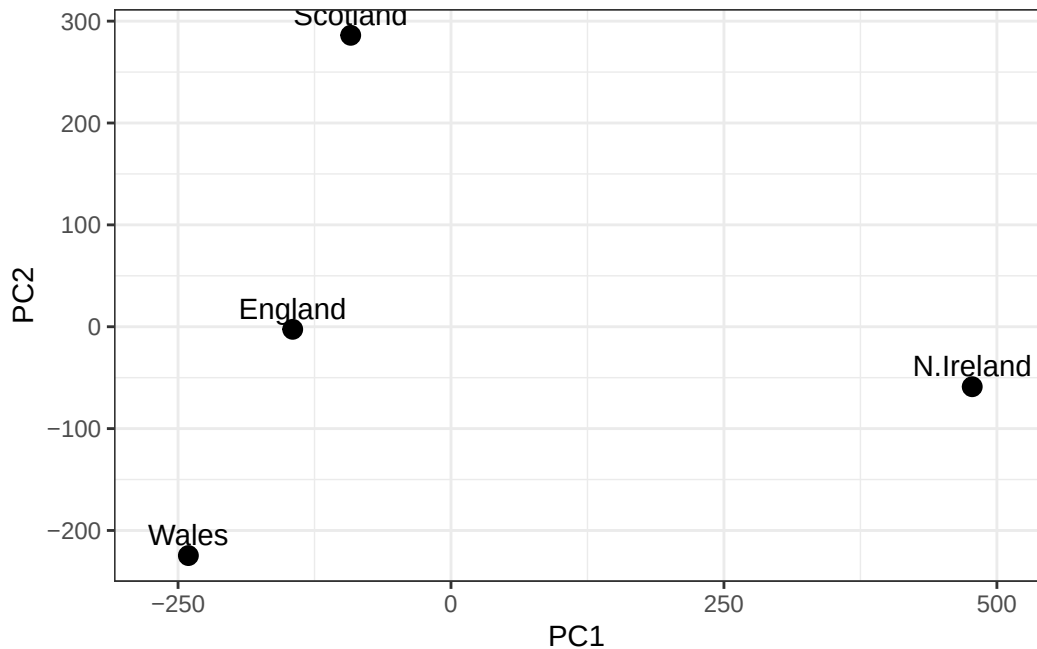
Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	2.921e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

Q7. Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

```
# Create a data frame for plotting
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

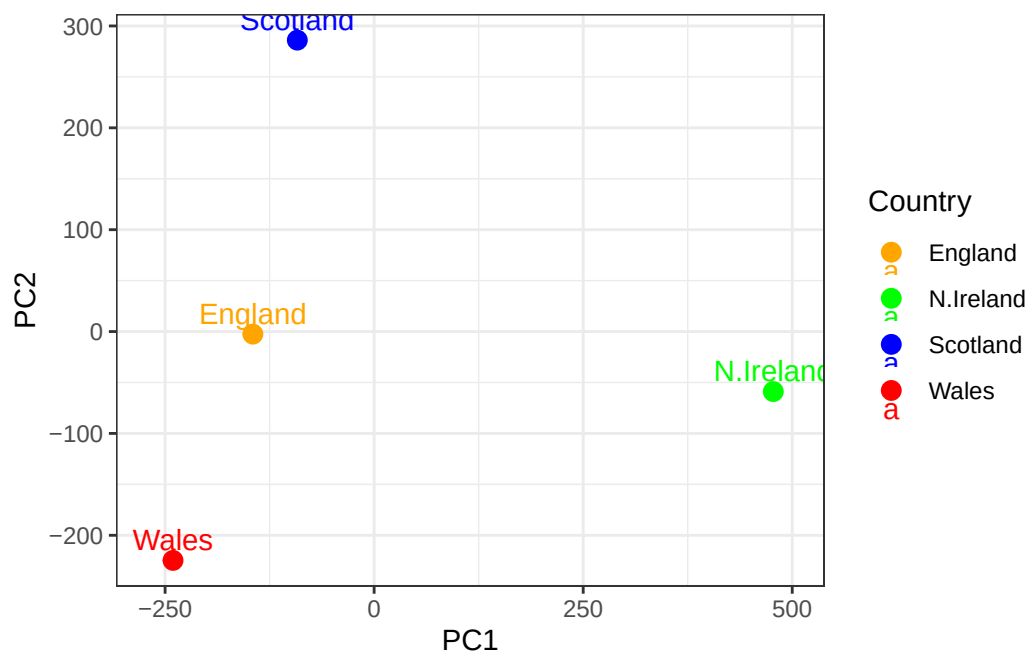
# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



Q8. Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.

```
ggplot(df) +
  aes(x = PC1, y = PC2, label = Country, color = Country) +
  geom_point(size = 3) +
```

```
geom_text(vjust = -0.5) +
scale_color_manual (values = c(
  "England" = "orange",
  "Wales" = "red",
  "Scotland" = "blue",
  "N.Ireland" = "green"
)) +
xlim (-270, 500) +
xlab("PC1") +
ylab ("PC2") +
theme_bw()
```



```
#Using pca$sdev to calculate how much variation in the original data each PC accounts for
v <- round( pca$sdev^2/sum(pca$sdev^2) * 100)
v
```

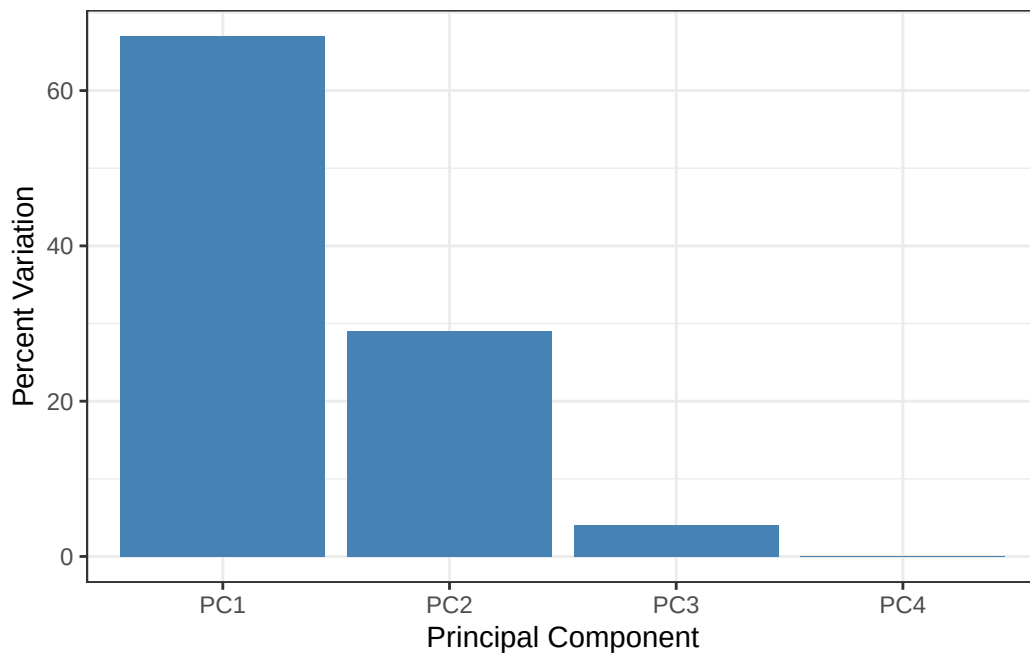
```
[1] 67 29 4 0
```

```
#Generating standard deviation, proportion of variance
#and cumulative proportion
z <- summary(pca)
z$importance
```


	PC1	PC2	PC3	PC4
Standard deviation	324.15019	212.74780	73.87622	2.921348e-14
Proportion of Variance	0.67444	0.29052	0.03503	0.000000e+00
Cumulative Proportion	0.67444	0.96497	1.00000	1.000000e+00

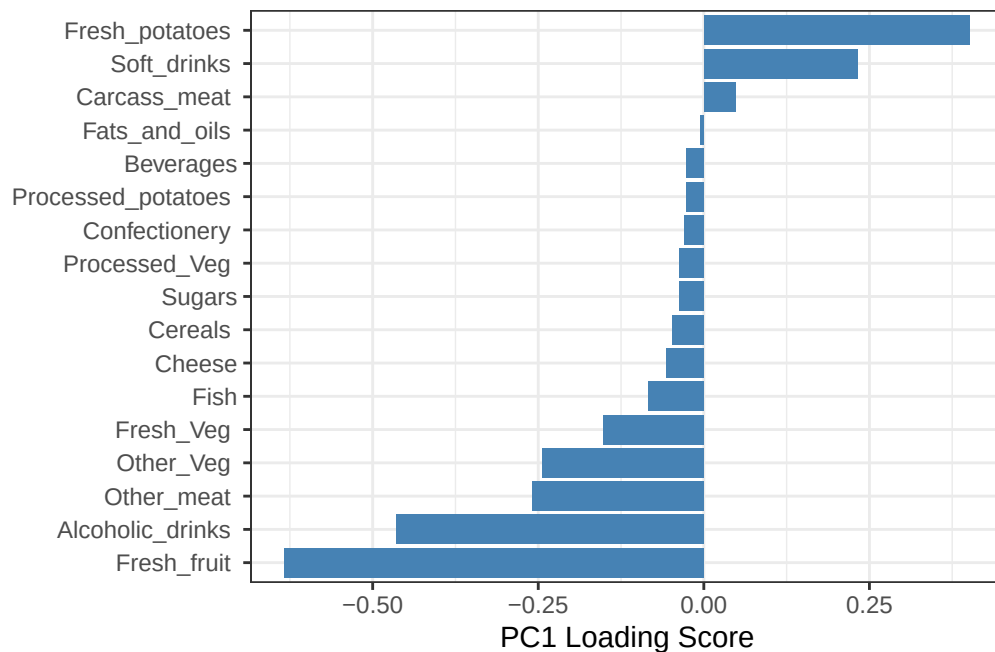
```
#Create scree plot with ggplot
variance_df <- data.frame(
  PC = factor(paste0("PC", 1:length(v)), levels = paste0("PC", 1:length(v))),
  Variance = v
)

ggplot(variance_df) +
  aes(x = PC, y = Variance) +
  geom_col(fill = "steelblue") +
  xlab("Principal Component") +
  ylab("Percent Variation") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 0))
```



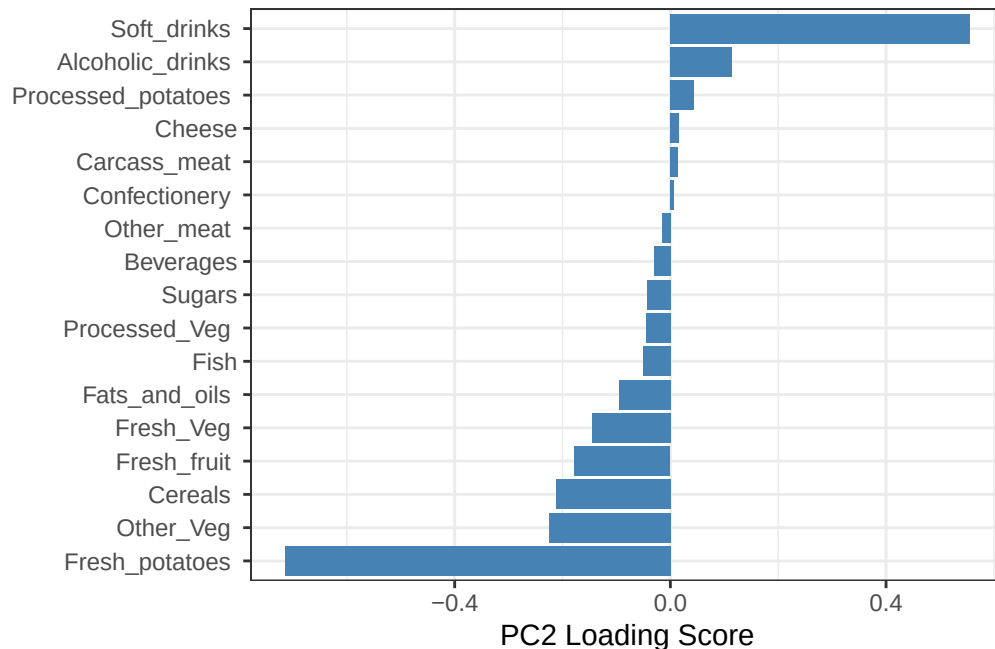
Digging deeper (variable loadings):

```
#Lets focus on PC1 as it accounts for >90% of variance
ggplot(pca$rotation) +
  aes(x = PC1,
      y = reorder(rownames(pca$rotation), PC1)) +
  geom_col(fill = "steelblue") +
  xlab("PC1 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



Q9: Generate a similar 'loadings plot' for PC2. What two food groups feature prominently and what does PC2 mainly tell us about?

```
#Generating a similar 'loadings plot' for PC2
ggplot(pca$rotation) +
  aes(x = PC2,
      y = reorder (rownames(pca$rotation), PC2)) +
  geom_col(fill = "steelblue") +
  xlab("PC2 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



The two food groups that feature most prominently in PC2 are soft drinks, as seen from the large positive loading, and the fresh potatoes, as seen from the large negative loading, and these two food groups have the largest loading magnitudes on opposite ends of the axis. This indicates that PC2 represents a high difference between diets high in soft drink consumption and diets high in fresh potato consumption.

2. PCA of RNA-seq data

```
url2 <- "https://tinyurl.com/expression-CSV"
rna.data <- read.csv(url2, row.names=1)
head(rna.data)
```

	wt1	wt2	wt3	wt4	wt5	ko1	ko2	ko3	ko4	ko5
gene1	439	458	408	429	420	90	88	86	90	93
gene2	219	200	204	210	187	427	423	434	433	426
gene3	1006	989	1030	1017	973	252	237	238	226	210
gene4	783	792	829	856	760	849	856	835	885	894
gene5	181	249	204	244	225	277	305	272	270	279
gene6	460	502	491	491	493	612	594	577	618	638

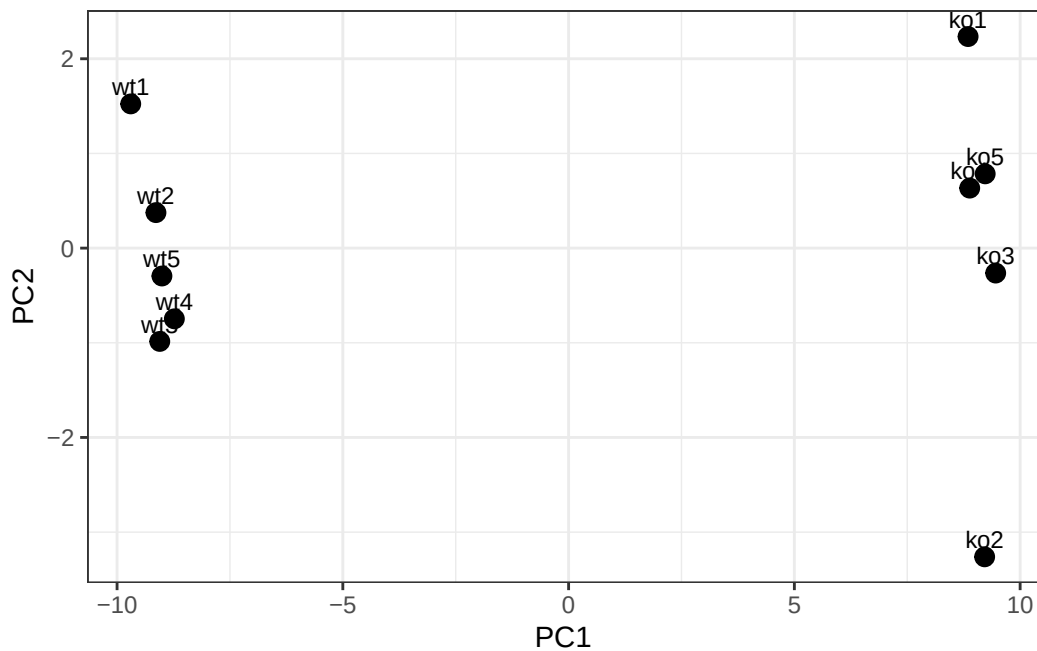
Q10: How many genes and samples are in this data set? How many PCs do you think it will take to have a useful overview of this data set (see below)?

There are 6 genes and 10 samples. Two PCs should generate a useful overview of this data set.

```
#Taking the transpose of the data
pca <- prcomp(t(rna.data), scale=TRUE)

#Creating data frame for plotting
df <- as.data.frame(pca$x)
df$Sample <- rownames(df)

#Plotting with ggplot
ggplot(df) +
  aes(x = PC1, y = PC2, label = Sample) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5, size = 3) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



```
#Generating summary of the PCA
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4	PC5	PC6	PC7
Standard deviation	9.6237	1.5198	1.05787	1.05203	0.88062	0.82545	0.80111
Proportion of Variance	0.9262	0.0231	0.01119	0.01107	0.00775	0.00681	0.00642
Cumulative Proportion	0.9262	0.9493	0.96045	0.97152	0.97928	0.98609	0.99251

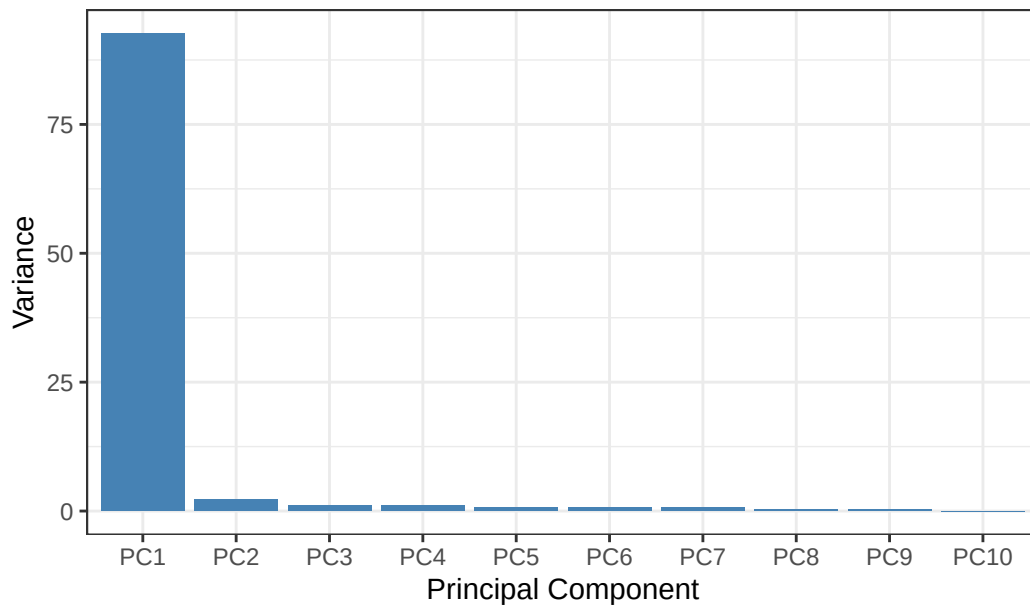
	PC8	PC9	PC10
Standard deviation	0.62065	0.60342	3.3e-15
Proportion of Variance	0.00385	0.00364	0.0e+00
Cumulative Proportion	0.99636	1.00000	1.0e+00

```
#Calculating variance
pca.var <- pca$sdev^2
pca.var.per <- round(pca.var/sum(pca.var)*100, 1)

#Generating scree plot data
scree_df <- data.frame(
  PC = factor(paste0("PC", 1:10), levels = paste0("PC", 1:10)),
  Variance = pca.var[1:10]
)

ggplot(scree_df) +
  aes(x = PC, y = Variance) +
  geom_col(fill = "steelblue") +
  ggtitle("Quick scree plot") +
  xlab("Principal Component") +
  ylab("Variance") +
  theme_bw()
```

Quick scree plot

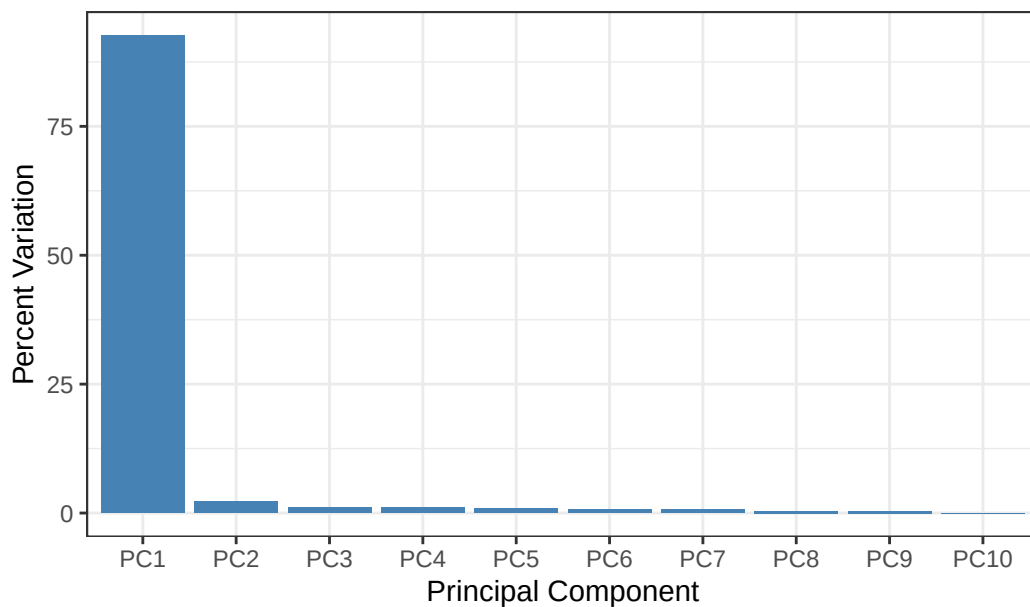


```
##Generating percentage variance  
pca.var.per
```

```
[1] 92.6  2.3  1.1  1.1  0.8  0.7  0.6  0.4  0.4  0.0
```

```
#Generating percent variance scree plot  
scree_pct_df <- data.frame(  
  PC = factor(paste0("PC", 1:10), levels = paste0("PC", 1:10)),  
  PercentVariation = pca.var.per[1:10]  
)  
  
ggplot(scree_pct_df) +  
  aes(x = PC, y = PercentVariation) +  
  geom_col(fill = "steelblue") +  
  ggtitle("Scree Plot") +  
  xlab("Principal Component") +  
  ylab("Percent Variation") +  
  theme_bw()
```

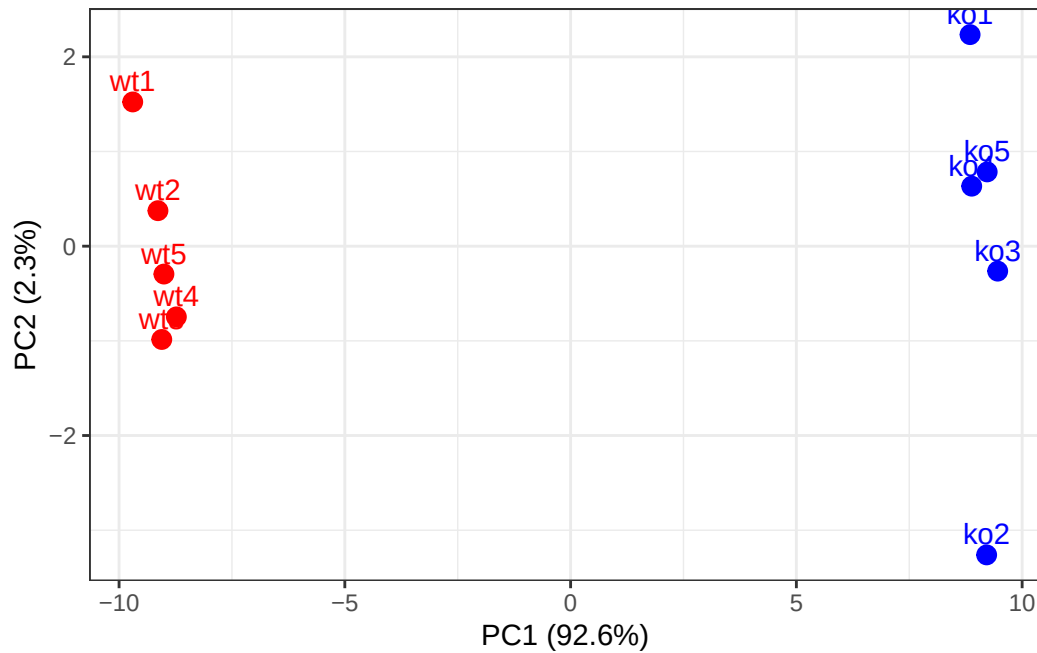
Scree Plot



```
#A vector of colors for wt and ko samples
colvec <- colnames(rna.data)
colvec[grep("wt", colvec)] <- "red"
colvec[grep("ko", colvec)] <- "blue"

#Adding condition to data frame
df$condition <- substr(df$Sample, 1, 2)
df$color <- colvec

ggplot(df) +
  aes(x = PC1, y = PC2, color = color, label = Sample) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5, hjust = 0.5, show.legend = FALSE) +
  scale_color_identity() +
  xlab(paste0("PC1 (", pca.var.per[1], "%)")) +
  ylab(paste0("PC2 (", pca.var.per[2], "%)")) +
  theme_bw()
```



Gene Loadings

```
loading_scores <- pca$rotation[,1]

#Viewing the top 10 measurements (genes) that contribute
# most to PC1 in either direction (+ or -)
gene_scores <- abs(loading_scores)
gene_score_ranked <- sort(gene_scores, decreasing=TRUE)

#Viewing the names of the top 10 genes
top_10_genes <- names(gene_score_ranked[1:10])
top_10_genes
```

```
[1] "gene100" "gene66" "gene45" "gene68" "gene98" "gene60" "gene21"
[8] "gene56" "gene10" "gene90"
```